# SISE GAMES 2026

SISE GAMING

Controller PROFILER

Modou Mboup
Yassine Cheniour
Nico Dena

# SOMMAIRE


01 PRÉSENTATION DU PROJET (OVERVIEW)

02 PRÉSENTATION JEUX

03 ARCHITECTURE

04 MACHINE LEARNING

05 APP DASH

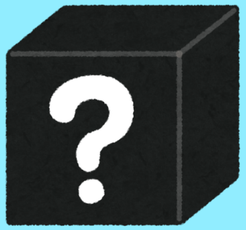
# PRÉSENTATION DE SISE GAMING

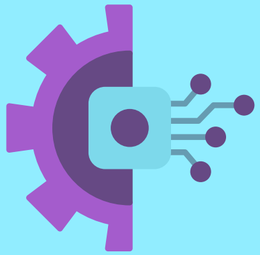
## Objectif du projet

- Profiler le comportement de jeu des joueurs
- Périphérique utilisé : Manette (fallback clavier)
- Capte les métriques en temps réel et agrége à la fin de la session

## Comment ?

- Capture en temps réel chaque intéraction entre le joueur et la manette
- Pressions de boutons, mouvements de joysticks, intensité des gâchettes
- Echantilloné à 30 HZ

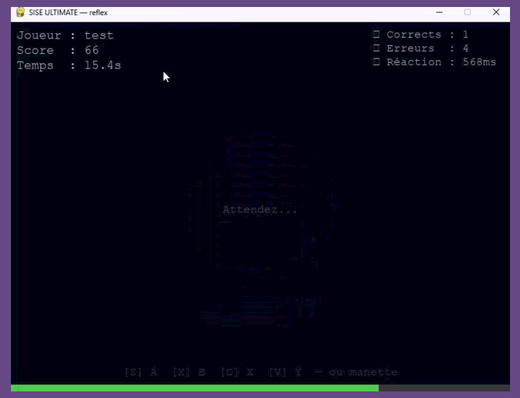




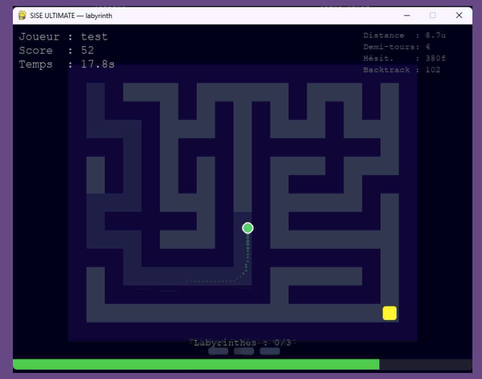
## Traitement

- Extraction de features
- Machine Learning : Clustering (UMAP + K-means)
- Analyse personnalisée (Mistral AI)

# PRÉSENTATION DES JEUX

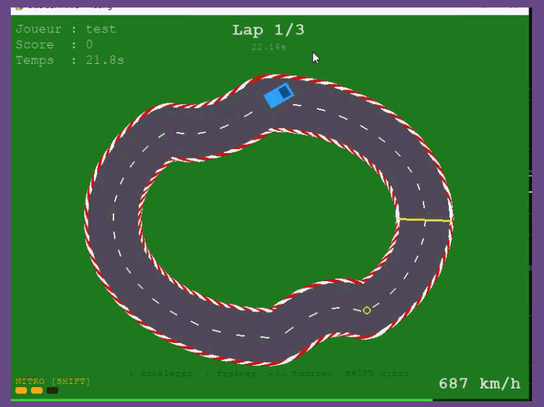



REFLEX




LABYRINTH

SHOOTER



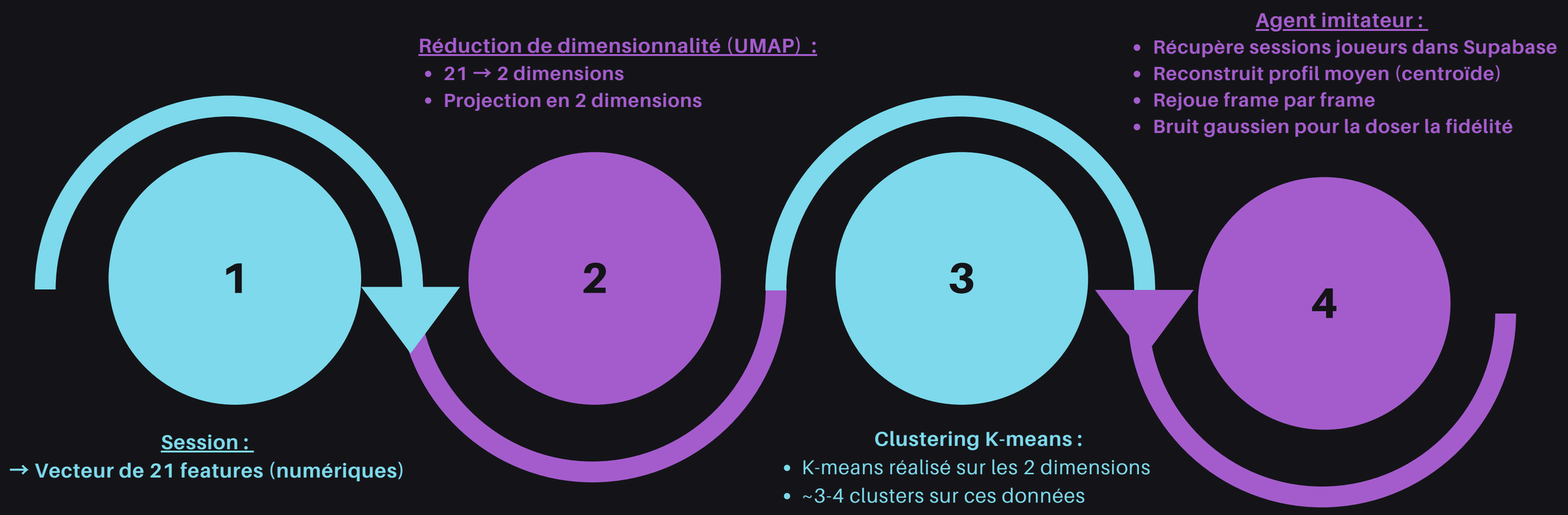

RACING

# ARCHITECTURE

L'application repose sur **4 couches** :

- **Couche périphérique** :
  - La manette est lue via Pygame à 30 Hz (30 frames par secondes)
  - A chaque frame, un objet contenant l'état instantanté de tous les axes et boutons est créé
- **Couche Capture** :
  - Un objet collecte tous les états pendant la durée de la session
  - A la fin, il agrége et calcule 21 features
  - Fréquence, variété des boutons, durée moyenne de maintien, ....
- **Couche Persistance** :
  - Dès la fin de session, les features sont envoyés ver Supabase cloud
  - Si supabase est indisponible (fallback csv)
  - Pendant la partie les frames sont aussi envoyés dans une table dédiée
- **Couche Analyse** :
  - Pipeline ML : Clustering de profil + dashboard
  - LLM Mistral : features +contexte = résumé JSON
  - Dashboard Dash : interroge Supabase toutes les 8 secondes

# MACHINE LEARNING




**Session :**

→ Vecteur de 21 features (numériques)

**Réduction de dimensionnalité (UMAP) :**

- 21 → 2 dimensions
- Projection en 2 dimensions

**Clustering K-means :**

- K-means réalisé sur les 2 dimensions
- ~3-4 clusters sur ces données

**Agent imitateur :**

- Récupère sessions joueurs dans Supabase
- Reconstruit profil moyen (centroïde)
- Rejoue frame par frame
- Bruit gaussien pour la doser la fidélité

# WEBAPP

plotly | Dash

**Spécificités:**
→ Architecture mono-fichier (app.py)
→ Rafraichissement automatique (toutes les 8s ou 3s pour Live)

MISTRAL AI_

**POINT D'ENTREE**

app.py

**Local (port : 8050)**

**PLUSIEURS PAGES**

- Live Session
- Profils
- Leaderboard (Classements)
- Résumés IA
- Chatbot

→ Lancement des jeux
→ Suivi en (quasi) live des actions

→ Projection UMAP (scatter plot)
→ Détails des individus
(Page longue à charger)

**Connexion à la BDD Supabase (.env)**

# PASSONS À LA DÉMO!